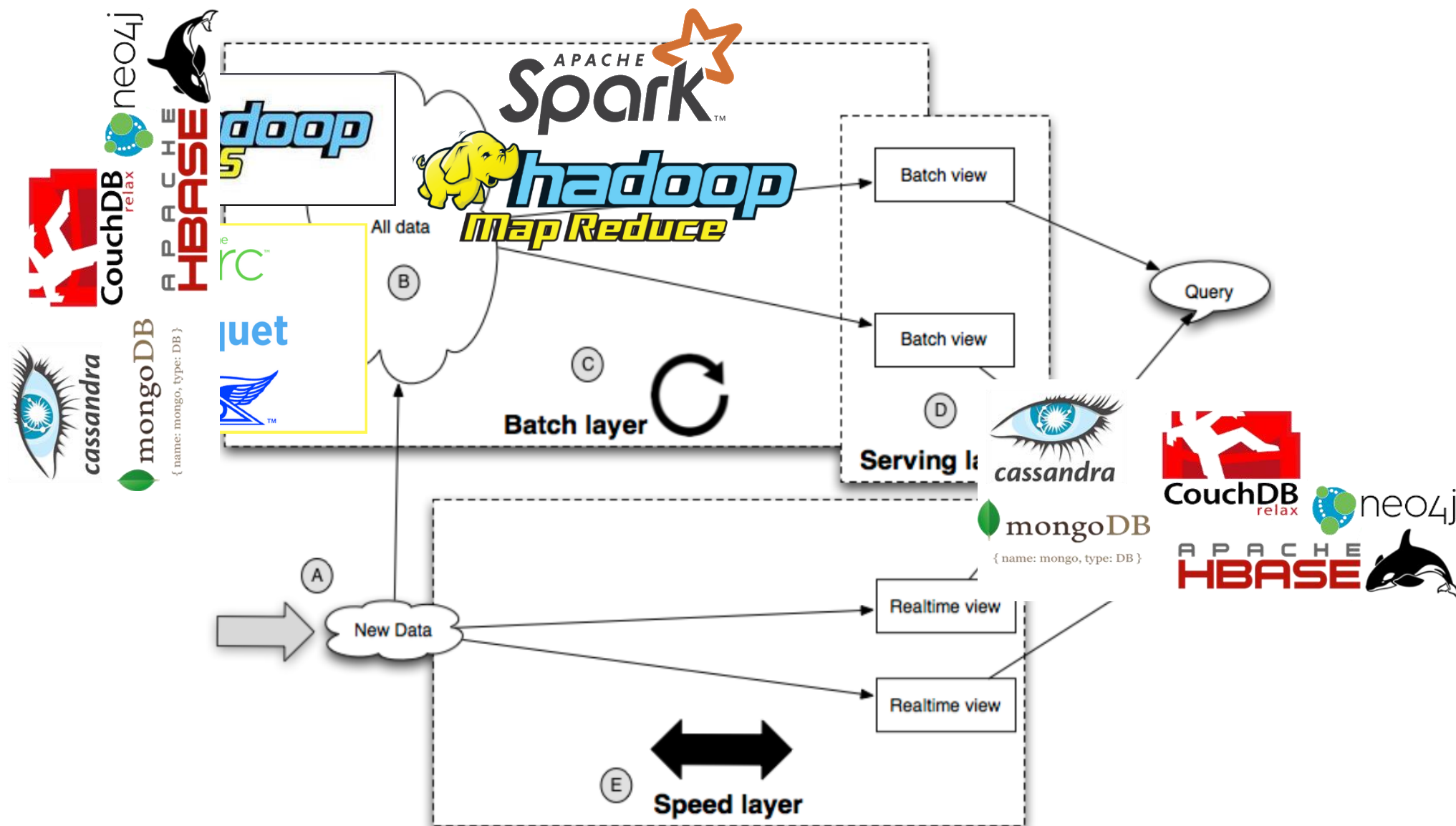




Information Systems for Big Data – 12th Lesson

Prof. G. Fenza gfenza@unisa.it

LAMBDA Architecture



Limitations of MapReduce



- Two-stage programming model
 - Workarounds are Cascading and Crunch
- Low-level programming API
 - Pig Latin
 - HiveQL
- Iterative applications
 - k-means clustering in Mahout refines its centroid location in every iteration
 - the output of each iteration needs to be materialized to HDFS
- Interactive analysis
 - Results of MapReduce jobs are available only at the end of execution.
- Strictly batch processing
 - Expect all the input to have been materialized before processing can start.
- Conflation between control and computation
 - Apache Yarn
 - Apache Mesos

Beyond Hadoop

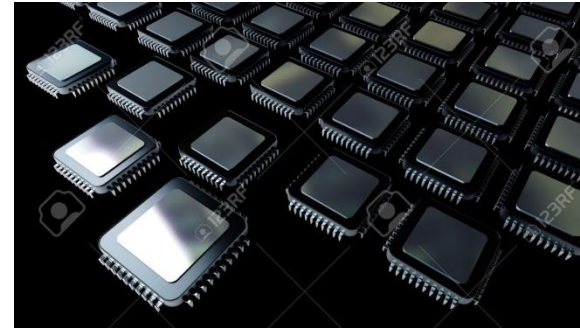


- *Interactive SQL*
 - Distributed query engine that uses dedicated “always on” daemons (like **Impala**) or container reuse (like **Hive** on Tez), low-latency responses for SQL queries on Hadoop while still scaling up to large dataset sizes.
 - Recently, **Spark2** introduced DataFrame programming model.
- *Iterative processing*
 - Many algorithms, such as those in machine learning, are iterative in nature, so it’s much more efficient to hold each intermediate working set in memory, compared to loading from disk on each iteration. The architecture of MapReduce does not allow this, but it’s straightforward with **Spark**, for example, and it enables a highly exploratory style of working with datasets.
- *Stream processing*
 - Streaming systems like **Kafka**, **Storm**, **Spark Streaming**, or **Samza** make it possible to run real-time, distributed computations on unbounded streams of data and emit results to Hadoop storage or external systems.
- *Search*
 - The **Solr** search platform can run on a Hadoop cluster, indexing documents as they are added to HDFS, and serving search queries from indexes stored in HDFS.

Hardware for Big Data



Lots of hard drives

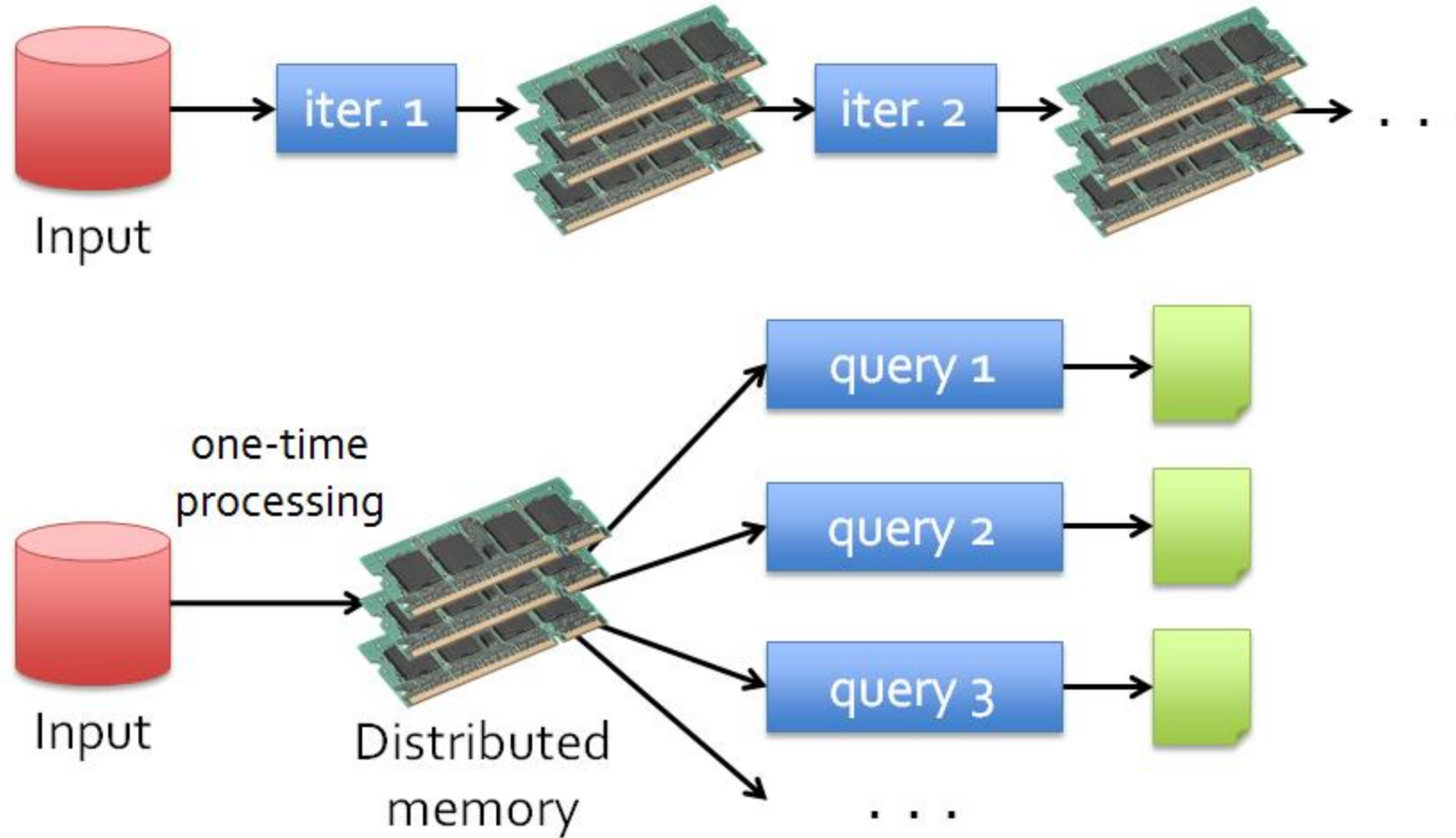


Lots of CPUs



And lots of memory!

Data Sharing in Spark Using RDD



10-100 × faster than network and disk

In-Memory Can Make a Big Difference



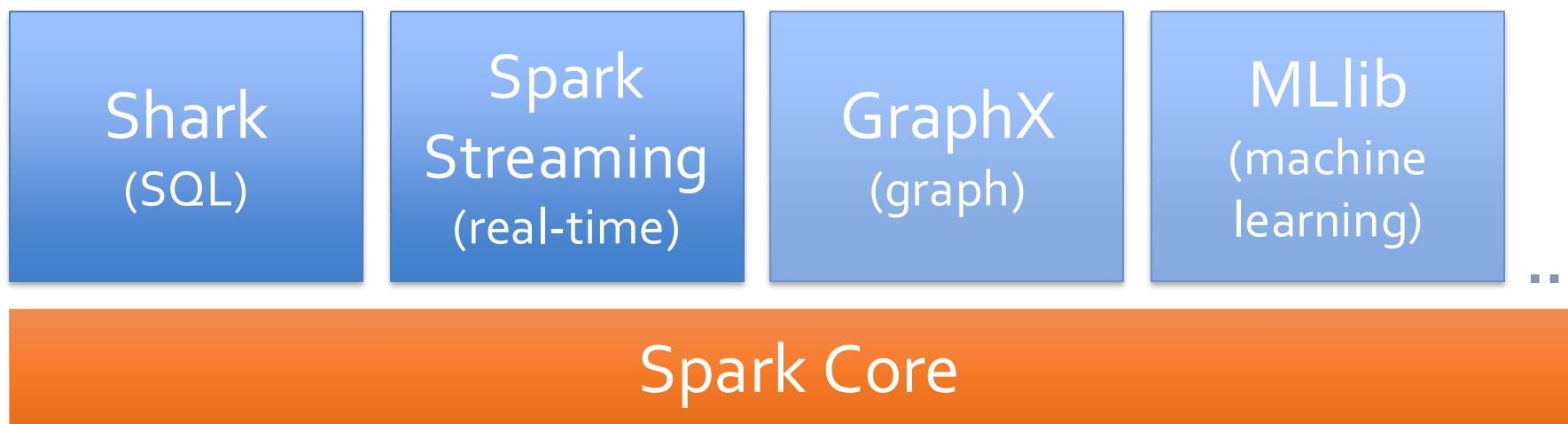
- Two iterative Machine Learning algorithms:



What is Spark



- Spark is the basis of a wide set of projects in the Berkeley Data Analytics Stack

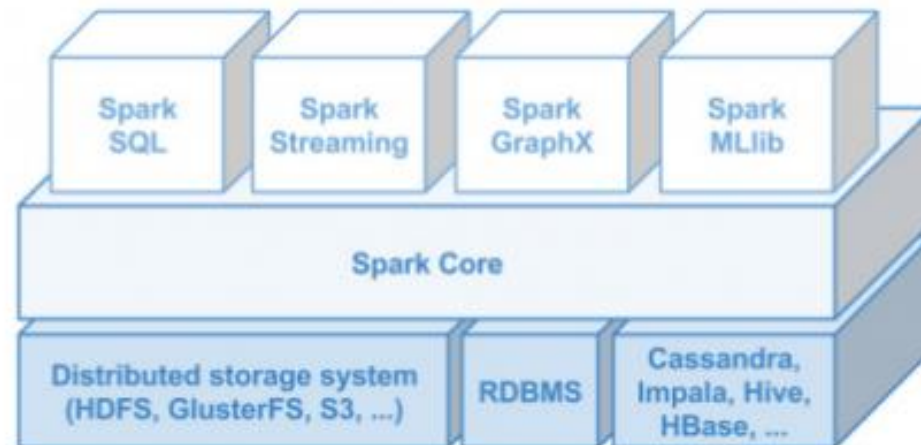


- Spark SQL (SQL on Spark)
- GraphX (graph processing)
- MLlib (machine learning library)
- Spark Streaming (stream processing)
- ...

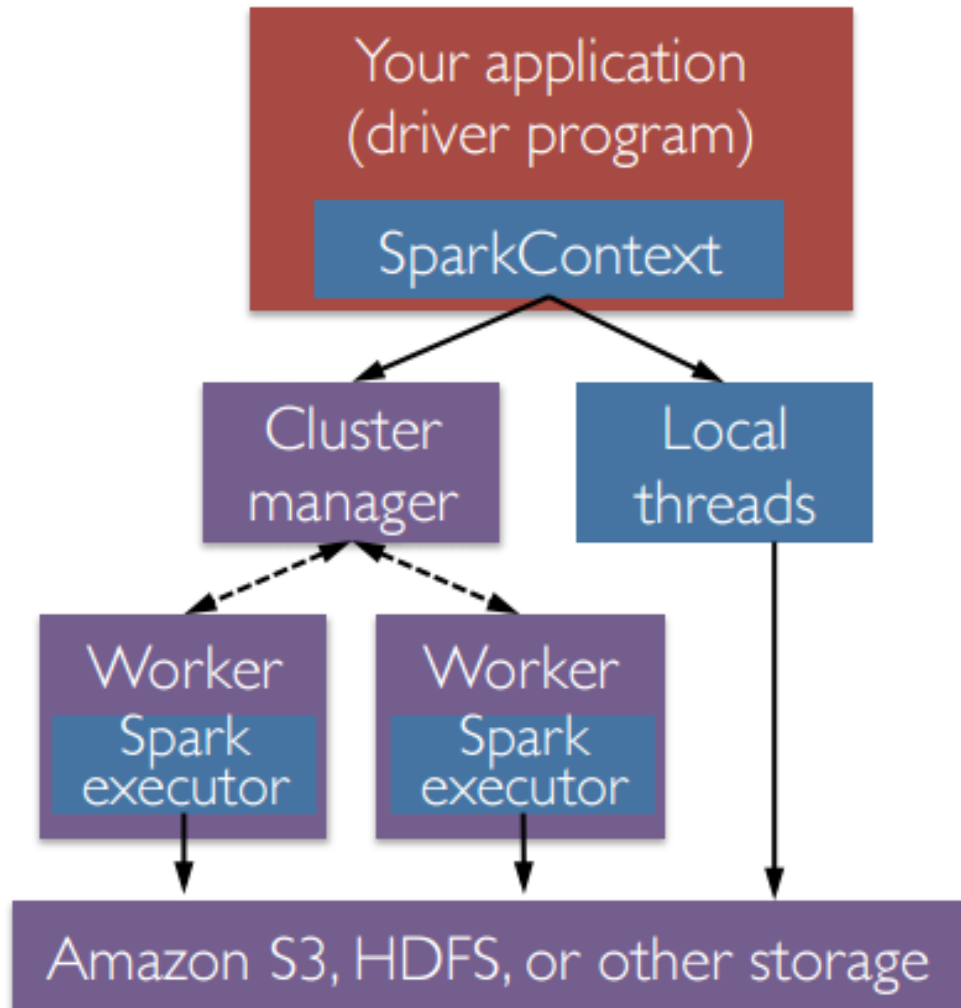
Spark Data Sources



- You can think of Spark as a flexible alternative to MapReduce
- Spark can use data stored in a variety of formats
- Local Files
 - `file:///opt/httpd/logs/access_log`
- S3
- Hadoop Distributed Filesystem
 - Regular files, sequence files, any other Hadoop InputFormat
- HBase, Cassandra, etc.



Spark Workflow



- A Spark program first creates a SparkContext object
 - Tells Spark how and where to access a cluster
 - Connect to several types of cluster managers (e.g., YARN, Mesos, or its own manager)
- Cluster manager:
 - Allocate resources across applications
- Spark executor:
 - Run computations
 - Access data storage

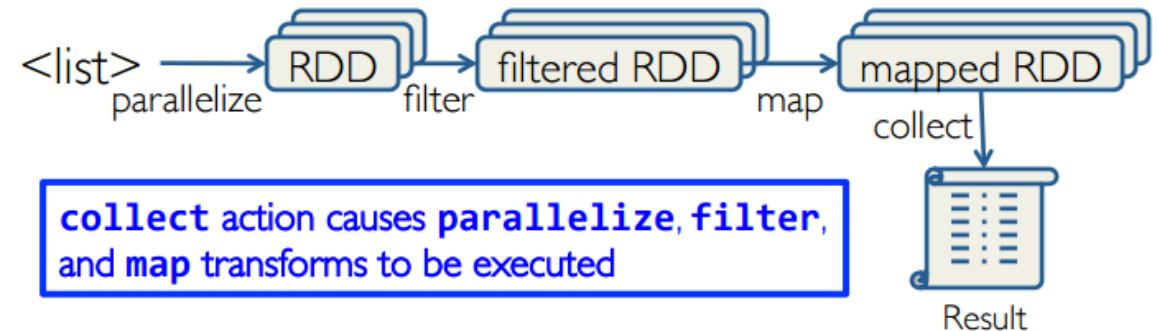
- At the core of Spark is the idea of a Resilient Distributed Dataset (RDD)
- Resilient Distributed Dataset (RDD) has 4 main features:
 - Distributed Collection of Data
 - Fault-tolerant
 - Parallel operation-partioned
 - Ability to use many data sources

- RDDs are immutable, lazily evaluated, and cacheable
- There are two types of Spark operations:
 - Transformations
 - Actions
- **Transformations** are basically a recipe to follow.
- **Actions** actually perform what the recipe says to do and returns something back.

Working with RDDs



- Create an RDD from a data source
 - by parallelizing existing collections (lists or arrays)
 - by transforming an existing RDDs
 - from files in HDFS or any other storage system
- Apply transformations to an RDD: e.g., map, filter
- Apply actions to an RDD: e.g., collect, count
- Users can control two other aspects:
 - Persistence
 - Partitioning



Creating RDDs



- From HDFS, text files, Amazon S3, Apache HBase, SequenceFiles, any other Hadoop InputFormat
- Creating an RDD from a File
 - `val inputfile = sc.textFile("...", 4)`
 - ❖ RDD distributed in 4 partitions
 - ❖ Elements are lines of input
 - ❖ Lazy evaluation means no execution happens now

```
scala> val inputfile = sc.textFile("pg100.txt")
inputfile: org.apache.spark.rdd.RDD[String] = pg100.txt MapPartitionsRDD[17] at
textFile at <console>:24
```

- Turn a collection into an RDD
 - `sc.parallelize([1, 2, 3])`, creating from a Python list
 - `sc.parallelize(Array("hello", "spark"))`, creating from a Scala Array
- Creating an RDD from an existing Hadoop InputFormat
 - `sc.hadoopFile(keyClass, valClass, inputFmt, conf)`

Spark Transformations



- Create new datasets from an existing one
- Use lazy evaluation: results not computed right away – instead Spark remembers set of transformations applied to base dataset
 - Spark optimizes the required calculations
 - Spark recovers from failures
- Some transformation functions

Transformation	Description
<code>map(func)</code>	return a new distributed dataset formed by passing each element of the source through a function <i>func</i>
<code>filter(func)</code>	return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true
<code>distinct([numTasks])</code>	return a new dataset that contains the distinct elements of the source dataset
<code>flatMap(func)</code>	similar to map, but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item)

Spark Actions



- Cause Spark to execute recipe to transform source
- Mechanism for getting results out of Spark
- Some action functions

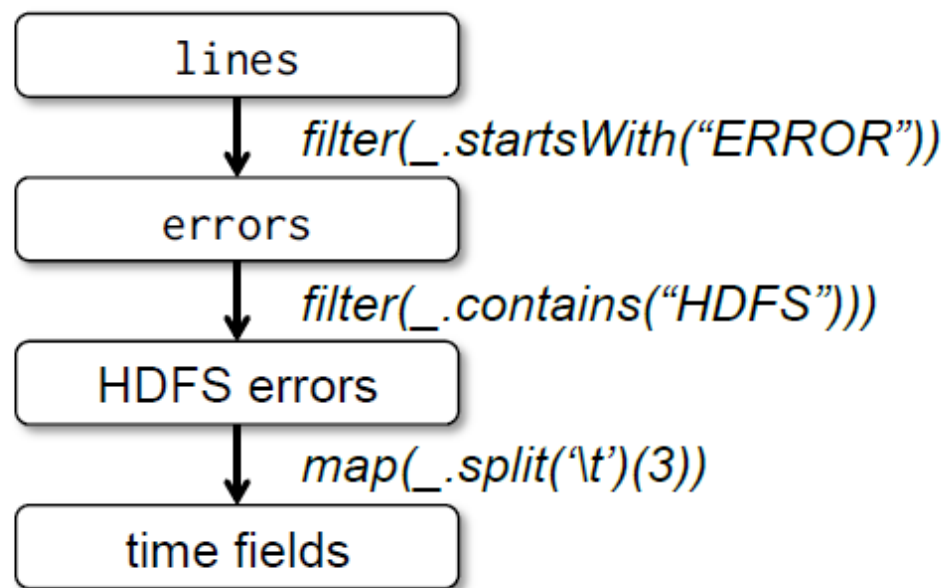
Action	Description
<code>reduce(func)</code>	aggregate dataset's elements using function <i>func</i> . <i>func</i> takes two arguments and returns one, and is commutative and associative so that it can be computed correctly in parallel
<code>take(n)</code>	return an array with the first <i>n</i> elements
<code>collect()</code>	return all the elements as an array WARNING: make sure will fit in driver program
<code>takeOrdered(n, key=func)</code>	return <i>n</i> elements ordered in ascending order or as specified by the optional key function

- Example: `words.collect().foreach(println)`

Example



- Web service is experiencing errors and an operators want to search terabytes of logs in the Hadoop file system to find the cause.



//base RDD

```
val lines = sc.textFile("hdfs://...")
```

//Transformed RDD

```
val errors = lines.filter(_.startsWith("Error"))
```

```
errors.persist()
```

```
errors.count()
```

```
errors.filter(_.contains("HDFS"))
```

```
.map(_.split('\t')(3))
```

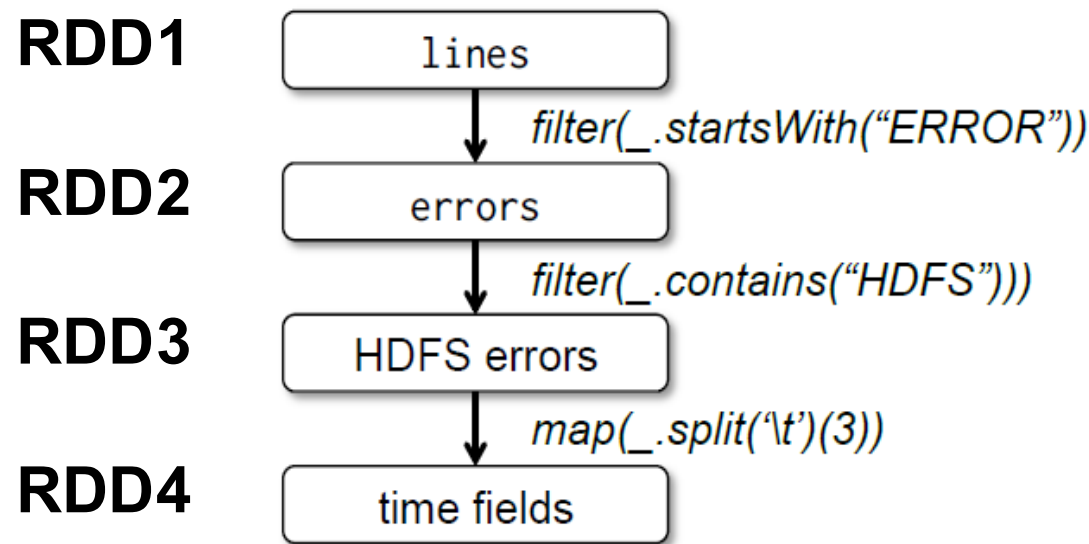
```
.collect()
```

- **Line1:** RDD backed by an HDFS file (base RDD lines not loaded in memory)
- **Line3:** Asks for errors to persist in memory (errors are in RAM)

Lineage Graph

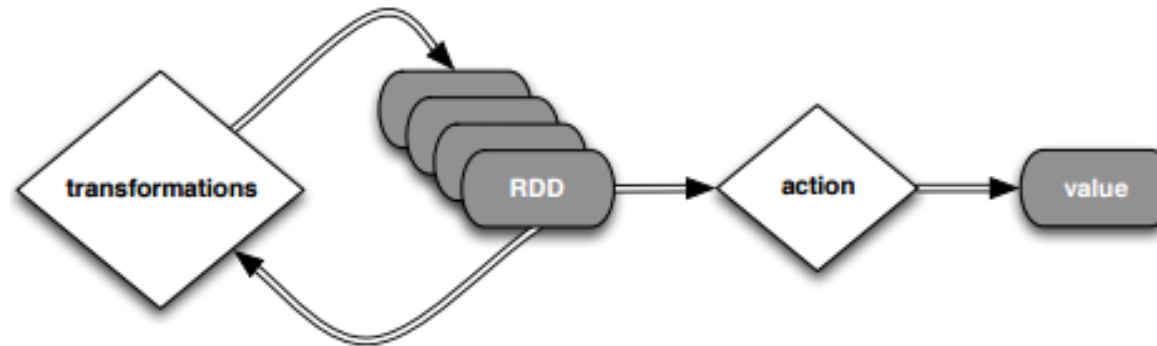


- RDD has enough information about how it was derived from to compute its partitions from data in stable storage.



- Example:
 - If a partition of errors is lost, Spark rebuilds it by applying a filter on only the corresponding partition of lines.
 - Partitions can be recomputed in parallel on different nodes, without having to roll back the whole program.

Deconstructed



//base RDD

```
val lines = sc.textFile("hdfs://...")
```

//Transformed RDD

```
val errors = lines.filter(_.startsWith("Error"))
```

```
errors.persist()
```

```
errors.count()
```

```
errors.filter(_.contains("HDFS"))
```

```
  .map(_.split('\t')(3))
```

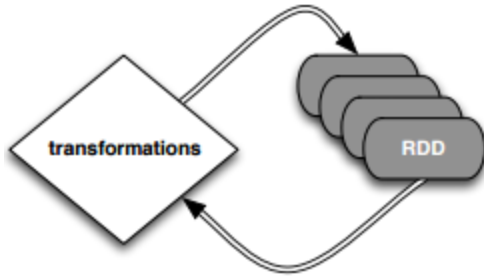
```
  .collect()
```

Deconstructed



//base RDD

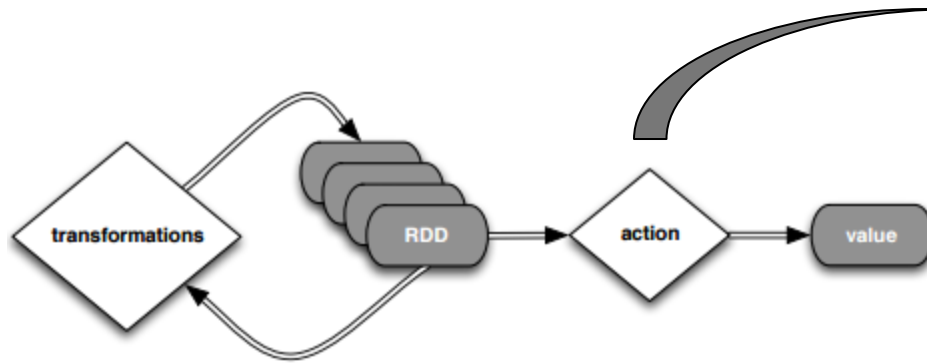
```
val lines = sc.textFile("hdfs://...")
```



//Transformed RDD

```
val errors = lines.filter(_.startsWith("Error"))
```

```
errors.persist()
```



errors.count()

count() causes Spark to: 1) read data; 2) sum within partitions; 3) combine sums in driver

Put transform and action together: *errors.filter(_.contains("HDFS")).map(_split('\t')(3)).collect()*

DataFrames



- A *DataFrame* is organized into named columns
 - Provides the benefits of RDDs (e.g., strong typing) with the benefits of Spark SQL's optimized execution engine
 - Can be constructed from JVM objects and then manipulated using functional transformations (map, flatMap, etc.)
 - The Dataset API is available in Scala and Java
 - It is conceptually equivalent to a table in a relational database or a data frame in R/Python, but with richer optimizations
 - An abstraction for selecting, filtering, aggregating and plotting structured data
 - In Scala and Java, a *DataFrame* is represented by a Dataset of Rows
 - ❖ Scala: *DataFrame* is simply a type alias of `Dataset[Row]`
 - ❖ Java: use `Dataset<Row>` to represent a *DataFrame*

Difference between DataFrame and RDD



- DataFrame more like a traditional database of two-dimensional form, in addition to data, but also to grasp the structural information of the data, that is, schema

Person	Name	Age	Height
Person	String	Int	Double
Person	String	Int	Double
Person	String	Int	Double
Person	String	Int	Double
Person	String	Int	Double
Person	String	Int	Double
Person	String	Int	Double

RDD[Person] DataFrame

- RDD[Person] although with Person for type parameters, but the Spark framework itself does not understand internal structure of Person class
- DataFrame has provided a detailed structural information, making Spark SQL can clearly know what columns are included in the dataset, and what is the name and type of each column. Thus, Spark SQL query optimizer can target optimization

RDD vs DataFrame



When to use RDDs?

- you want low-level transformation and actions and control on your dataset;
- your data is unstructured, such as media streams or streams of text;
- you want to manipulate your data with functional programming constructs than domain specific expressions;
- you don't care about imposing a schema, such as columnar format, while processing or accessing data attributes by name or column;
- you can forgo some optimization and performance benefits available with DataFrames and Datasets for structured and semi-structured data.



Hands on PySpark

- DataFrames

Hands on PySpark via Docker



- `docker pull jupyter/all-spark-notebook`
- `docker run -it --rm -p 8888:8888 -p 4040:4040 -v [YOUR_PATH]:/home/jovyan/work jupyter/all-spark-notebook`

Hands on PySpark via Colab



- `!apt-get install openjdk-8-jdk-headless -qq > /dev/null`
- `!wget -q https://archive.apache.org/dist/spark/spark-3.0.1/spark-3.0.1-`
- `bin-hadoop2.7.tgz`
- `!tar xf ./spark-3.0.1-bin-hadoop2.7.tgz !pip install -q findspark`

Hands on PySpark via PyCharm or other IDE



- Install JDK8 or 11
- Install Python3
- Install Spark
- Configure Environment Variables (e.g., JAVA_HOME, SPARK_HOME)
- Configure PySpark env setting the interpreter and installing pyspark modules

Start SparkSession



- From jupyter notebook

```
from pyspark.sql import SparkSession
```

```
# May take a little while on a local computer  
spark = SparkSession.builder.appName( "Basics" ).getOrCreate( )
```

- May take a little while on a local computer
- Then, you can open
 - <http://localhost:4040/jobs>

Spark Web Console



- You can browse the web interface for the information of Spark Jobs, storage, etc. at: <http://localhost:4040>



JobsStagesStorageEnvironmentExecutorsSQLSpark shell application UI

Spark Jobs (?)

User: comp9313
Total Uptime: 7.1 h
Scheduling Mode: FIFO
Completed Jobs: 46
[▶ Event Timeline](#)

Completed Jobs (46)

Job Id	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
45	take at <console>:28	2016/09/18 08:28:38	0.2 s	3/3	3/3
44	take at <console>:29	2016/09/18 08:20:39	0.3 s	3/3	3/3
43	reduce at <console>:29	2016/09/18 08:17:13	92 ms	2/2	2/2
42	collect at <console>:29	2016/09/18	68 ms	2/2	2/2

Spark DataFrame



- DataFrame
 - Basics
 - Operations
 - GroupBy and Aggregate
 - MissingData
 - Datetime
- https://spark.apache.org/docs/latest/api/python/getting_started/quickstart_ps.html

Read Data



```
# We'll discuss how to read other options later.  
# This dataset is from Spark's examples
```

```
# Might be a little slow locally  
df = spark.read.json('people.json')
```

```
# Note  
df.show()
```

```
df.printSchema()
```

```
df.coalesce(1).write.parquet('data')
```

- spark.read.csv
- spark.read.format
- spark.read.jdbc
- spark.read.json
- spark.read.load
- spark.read.option
- spark.read.options
- spark.read.orc
- spark.read.parquet
- spark.read.schema

Read Data



```
# Let Spark know about the header and infer the Schema types!  
df = spark.read.csv('appl_stock.csv',inferSchema=True,header=True)
```

```
df.pr  
spark.read.csv  
spark.read.format  
spark.read.jdbc  
spark.read.json  
spark.read.load  
spark.read.option  
spark.read.options  
spark.read.orc  
spark.read.parquet  
spark.read.schema  
ow()  
"))  
)
```

DataFrame Schema



```
df = spark.read.json('people.json')
```

```
# Note how data is missing!  
df.show()
```

```
+-----+-----+  
|  age |   name |  
+-----+-----+  
| null | Michael |  
|   30 |    Andy |  
|   19 |   Justin |  
+-----+-----+
```

```
df.printSchema()
```

```
root  
|-- age: long (nullable = true)  
|-- name: string (nullable = true)
```

DataFrame Schema



- `df.printSchema()`

```
# Let Spark know about the header and infer the Schema types!  
df = spark.read.csv('appl_stock.csv',inferSchema=True,header=True)
```

```
df.printSchema()
```

```
root
```

```
|-- Date: timestamp (nullable = true)  
|-- Open: double (nullable = true)  
|-- High: double (nullable = true)  
|-- Low: double (nullable = true)  
|-- Close: double (nullable = true)  
|-- Volume: integer (nullable = true)  
|-- Adj Close: double (nullable = true)
```

`.option("timestampFormat",
"yyyy-MM-dd")`

DataFrame Schema



- `df.printSchema()`

```
# Let Spark know about the header and infer the Schema types!  
df = spark.read.csv('appl_stock.csv', inferSchema=True, header=False)
```

```
df.printSchema()
```

```
root
```

```
|-- _c0: string (nullable = true)  
|-- _c1: string (nullable = true)  
|-- _c2: string (nullable = true)  
|-- _c3: string (nullable = true)  
|-- _c4: string (nullable = true)  
|-- _c5: string (nullable = true)  
|-- _c6: string (nullable = true)
```

DataFrame Schema



- if you don't like inferred schema

```
from pyspark.sql.types import StructField,  
                                StringType,  
                                IntegerType, StructType
```

```
data_schema = [StructField("age", IntegerType(), True),  
               StructField("name", StringType(), True)]
```

```
final_struct = StructType(fields=data_schema)
```

```
df = spark.read.json('people.json', schema=final_struct)
```

DataFrame Show, Head, Columns



```
df = spark.read.json('people.json')
```

```
# Note how data is missing!  
df.show()
```

```
+-----+-----+  
|  age |   name |  
+-----+-----+  
| null | Michael |  
|   30 |    Andy |  
|   19 |   Justin |  
+-----+-----+
```

```
df.head()
```

```
Row(age=None, name='Michael')
```

```
df.columns
```

```
['age', 'name']
```

DataFrame Show, Head, Columns



```
# Let Spark know about the header and infer the Schema types!  
df = spark.read.csv('appl_stock.csv',inferSchema=True,header=True)
```

```
df.head(2)
```

```
[Row(Date=datetime.datetime(2010, 1, 4, 0, 0), Open=213.429998, High=214.499996, Low=212.380000999999996, Close=214.009998, Volume=123432400, Adj Close=27.727039),  
 Row(Date=datetime.datetime(2010, 1, 5, 0, 0), Open=214.599998, High=215.589994, Low=213.249994, Close=214.379993, Volume=150476200, Adj Close=27.7749760000000002)]
```

```
df.columns
```

```
['Date', 'Open', 'High', 'Low', 'Close', 'Volume', 'Adj Close']
```

DataFrame describe()



```
df = spark.read.json('people.json')
```

```
# Note how data is missing!  
df.show()
```

```
+-----+-----+  
|  age |   name |  
+-----+-----+  
| null | Michael |  
|   30 |    Andy |  
|   19 |   Justin |  
+-----+-----+
```

```
df.describe().show()
```

```
+-----+-----+-----+  
| summary |          age |    name |  
+-----+-----+-----+  
|    count |              2 |        3 |  
|     mean |            24.5 |     null |  
|    stddev | 7.7781745930520225 |     null |  
|      min |             19 |    Andy |  
|      max |             30 | Michael |  
+-----+-----+-----+
```

DataFrame Select and Column-based Operations



```
df['age']
```

```
Column<b'age'>
```

```
type(df['age'])
```

```
pyspark.sql.column.Column
```

```
df.select('age')
```

```
DataFrame[age: bigint]
```

```
type(df.select('age'))
```

```
pyspark.sql.dataframe.DataFrame
```

```
df.select('age').show()
```

```
+----+
| age|
+----+
| null|
|  30|
|  19|
+----+
```

```
df.withColumn('doubleage',df['age']*2).show()
```

```
+----+-----+-----+
| age|  name|doubleage|
+----+-----+-----+
| null|Michael|      null|
|  30|   Andy|       60|
|  19|  Justin|       38|
+----+-----+-----+
```

```
df.withColumnRenamed('age','supernewage').show()
```

```
+-----+-----+
|supernewage|  name|
+-----+-----+
|          null|Michael|
|          30|   Andy|
|          19|  Justin|
+-----+-----+
```

SQL-like interaction



```
# Register the DataFrame as a SQL temporary view  
df.createOrReplaceTempView("people")
```

```
sql_results = spark.sql("SELECT * FROM people")
```

```
sql_results
```

```
DataFrame[age: bigint, name: string]
```

```
sql_results.show()
```

```
+----+-----+  
| age |    name |  
+----+-----+  
| null | Michael |  
|   30 |    Andy |  
|   19 |   Justin |  
+----+-----+
```

DataFrame Filtering



```
# Make sure to add in the parenthesis separating the statements!  
df2 = df.filter( (df["Close"] < 200) & (df['Open'] > 200) )  
df2.select(['Date', 'Open', 'Close']).show()
```

Date	Open	Close
2010-01-22 00:00:00	206.78000600000001	197.75
2010-01-28 00:00:00	204.930004	199.289995
2010-01-29 00:00:00	201.079996	192.060003

From DataFrame to Dict



```
result = df.filter(df["Low"] == 197.16).collect()
```

```
# Note the nested structure returns a nested row object  
type(result[0])
```

```
pyspark.sql.types.Row
```

```
row = result[0]
```

```
row.asDict()
```

```
{'Adj Close': 25.620401,  
 'Close': 197.75,  
 'Date': datetime.datetime(2010, 1, 22, 0, 0),  
 'High': 207.499996,  
 'Low': 197.16,  
 'Open': 206.78000600000001,  
 'Volume': 220441900}
```

```
result[0].asDict()['Close']
```

```
197.75
```

Missing Data - Drop



- You can use the `.na` functions for missing data.
- The `drop` command has the following parameters:
 - `df.na.drop(how='any', thresh=None, subset=None)`
 - * param **how**: 'any' or 'all'.
If 'any', drop a row if it contains any nulls. If 'all', drop a row only if all its values are null.
 - * param **thresh**: int, default None
If specified, drop rows that have **less than `thresh` non-null values**. This overwrites the `how` parameter.
 - * param **subset**: list of column names to consider.

Missing Data - Drop



```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | null |
| emp2 | null | null |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

ContainsNull.csv

```
# Drop any row that contains missing data
df.na.drop().show()
```

```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

```
df.na.drop(how='any').show()
```

```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

Missing Data - Drop



Id	Name	Sales
emp1	John	null
emp2	null	null
emp3	null	345.0
emp4	Cindy	456.0

ContainsNull.csv

```
df.na.drop(how='all').show()
```

Id	Name	Sales
emp1	John	null
emp2	null	null
emp3	null	345.0
emp4	Cindy	456.0

Missing Data - Drop



```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | null |
| emp2 | null | null |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

ContainsNull.csv

```
# Has to have at least 2 NON-null values
df.na.drop(thresh=2).show()
```

```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | null |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

Missing Data - Drop



Id	Name	Sales
emp1	John	null
emp2	null	null
emp3	null	345.0
emp4	Cindy	456.0

ContainsNull.csv

```
df.na.drop(subset=["Sales"]).show()
```

Id	Name	Sales
emp3	null	345.0
emp4	Cindy	456.0

Missing Data - Fill



```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | null |
| emp2 | null | null |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

ContainsNull.csv

```
df.na.fill('NEW VALUE').show()
```

```
+-----+-----+-----+
|  Id |      Name | Sales |
+-----+-----+-----+
| emp1 |      John | null |
| emp2 | NEW VALUE | null |
| emp3 | NEW VALUE | 345.0 |
| emp4 |      Cindy | 456.0 |
+-----+-----+-----+
```

Missing Data - Fill



```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | null |
| emp2 | null | null |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

ContainsNull.csv

```
df.na.fill(0).show()
```

```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | 0.0 |
| emp2 | null | 0.0 |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

Missing Data - Fill



```
+-----+-----+-----+
|  Id | Name | Sales |
+-----+-----+-----+
| emp1 | John | null |
| emp2 | null | null |
| emp3 | null | 345.0 |
| emp4 | Cindy | 456.0 |
+-----+-----+-----+
```

ContainsNull.csv

```
df.na.fill('No Name', subset=['Name']).show()
```

```
+-----+-----+-----+
|  Id |   Name | Sales |
+-----+-----+-----+
| emp1 |   John | null |
| emp2 | No Name | null |
| emp3 | No Name | 345.0 |
| emp4 |   Cindy | 456.0 |
+-----+-----+-----+
```

Missing Data - Fill



```
+-----+-----+-----+
|  Id|  Name|Sales|
+-----+-----+-----+
|emp1|  John| null|
|emp2|  null| null|
|emp3|  null|345.0|
|emp4|Cindy|456.0|
+-----+-----+-----+
```

ContainsNull.csv

```
from pyspark.sql.functions import mean
mean_val = df.select(mean(df['Sales'])).collect()

# Weird nested formatting of Row object!
mean_val[0][0]

400.5
```

```
mean_sales = mean_val[0][0]
```

```
df.na.fill(mean_sales, ["Sales"]).show()
```

```
+-----+-----+-----+
|  Id|  Name|Sales|
+-----+-----+-----+
|emp1|  John|400.5|
|emp2|  null|400.5|
|emp3|  null|345.0|
|emp4|Cindy|456.0|
+-----+-----+-----+
```

Missing Data - Fill



```
+-----+-----+-----+
|  Id|  Name|Sales|
+-----+-----+-----+
|emp1|  John| null|
|emp2|  null| null|
|emp3|  null|345.0|
|emp4|Cindy|456.0|
+-----+-----+-----+
```

ContainsNull.csv

```
from pyspark.sql.functions import mean
```

```
# One (very ugly) one-liner
df.na.fill(df.select(mean(df['Sales']))).collect()[0][0],['Sales']).show()
```

```
+-----+-----+-----+
|  Id|  Name|Sales|
+-----+-----+-----+
|emp1|  John|400.5|
|emp2|  null|400.5|
|emp3|  null|345.0|
|emp4|Cindy|456.0|
+-----+-----+-----+
```

DataFrame GroupBy

Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

Mean

```
df.groupby("Company").mean().show()
```

Company	avg(Sales)
APPL	370.0
GOOG	220.0
FB	610.0
MSFT	322.33333333333333

DataFrame GroupBy



Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

Max

```
df.groupby( "Company" ).max( ).show( )
```

Company	max(Sales)
APPL	750.0
GOOG	340.0
FB	870.0
MSFT	600.0

DataFrame GroupBy



Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

```
grouped = df.groupby("Company")
```

```
grouped.agg({"Sales": 'max'}).show()
```

Company	max(Sales)
APPL	750.0
GOOG	340.0
FB	870.0
MSFT	600.0

DataFrame Aggregation Operators



Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

```
# Max sales across everything  
df.agg({'Sales': 'max'}).show()
```

```
+-----+  
| max(Sales) |  
+-----+  
|      870.0 |  
+-----+
```

DataFrame Aggregation Operators



Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

```
from pyspark.sql.functions import countDistinct, avg, stddev
```

```
df.select(countDistinct("Sales")).show()
```

```
+-----+
|count(DISTINCT Sales)|
+-----+
|                    11|
+-----+
```

```
df.select(countDistinct("Sales").alias("Distinct Sales")).show()
```

```
+-----+
|Distinct Sales|
+-----+
|            11|
+-----+
```

```
df.select(avg('Sales')).show()
```

```
+-----+
|      avg(Sales) |
+-----+
|360.5833333333333|
+-----+
```

DataFrame Aggregation Operators



Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

```
from pyspark.sql.functions import format_number
```

```
sales_std = df.select(stddev("Sales").alias('std'))
```

```
sales_std.show()
```

```
+-----+
|                std|
+-----+
|250.08742410799007|
+-----+
```

```
# format_number("col_name", decimal places)
sales_std.select(format_number('std', 2)).show()
```

```
+-----+
|format_number(std, 2)|
+-----+
|                250.09|
+-----+
```

DataFrame OrderBy



Company	Person	Sales
GOOG	Sam	200.0
GOOG	Charlie	120.0
GOOG	Frank	340.0
MSFT	Tina	600.0
MSFT	Amy	124.0
MSFT	Vanessa	243.0
FB	Carl	870.0
FB	Sarah	350.0
APPL	John	250.0
APPL	Linda	130.0
APPL	Mike	750.0
APPL	Chris	350.0

sales_info.csv

```
df.orderBy(df["Sales"].desc()).show()
```

Company	Person	Sales
FB	Carl	870.0
APPL	Mike	750.0
MSFT	Tina	600.0
FB	Sarah	350.0
APPL	Chris	350.0
GOOG	Frank	340.0
APPL	John	250.0
MSFT	Vanessa	243.0
GOOG	Sam	200.0
APPL	Linda	130.0
MSFT	Amy	124.0
GOOG	Charlie	120.0

DataFrame Dates and Timestamps

- dayofmonth
- hour
- dayofyear
- month
- year
- weekofyear
- date_format

appl_stock.csv

```
newdf = df.withColumn("Year", year(df['Date']))
newdf.groupBy("Year").mean()[['avg(Year)', 'avg(Close)']].show()
```

avg(Year)	avg(Close)
2015.0	120.03999980555547
2013.0	472.6348802857143
2014.0	295.4023416507935
2012.0	576.0497195640002
2016.0	104.60400786904763
2010.0	259.84246000000002
2011.0	364.00432532142867

Exercise on DataFrame



- Open file "Spark DataFrames Project Exercise.ipynb"
- Try to solve the exercise on walmart_stock.csv

20 min

Exercise on DataFrame



- Open file "Spark DataFrames Project Exercise.ipynb"
- SOLUTION OF the exercise on walmart_stock.csv